

Exploration of How Different Types of Sparsity Affect Unary GEMM Units

Xinyu Liu

`liu3680@purdue.edu`

Exploration of How Different Types of Sparsity Affect Unary GEMM Units

Xinyu Liu

liu3680@purdue.edu

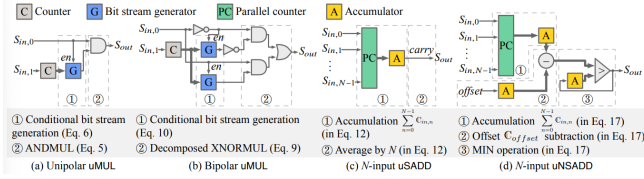


Figure 1: uGEMM functional units [1]

1 Introduction and Background

Stochastic computing offers an alternative to conventional binary arithmetic. Matrix multiplication is a dominant operation in AI workloads, and stochastic computing provides better area and power efficiency, at the tradeoff of accuracy or latency compared with binary arithmetic. In this report, four stochastic GEMM designs (uGEMM [1], tuGEMM [2], tubGEMM [3], and BISC-MVM [4]) are studied, exploring how different types of sparsity (bit-wise, matrix-wise, and column-wise) affect latency and accuracy in stochastic GEMM. While previous work [5] explores area and power for each design through synthesis, this report focuses on latency and accuracy.

This report documents a semester of work spent implementing, debugging, and comparing these four designs in a unified Python simulation environment, developed from UnarySim [6] and documented in [7].

2 Design

Four designs are used for comparison, and the architectural setup for each is described below. Since most modern AI workloads (such as MLP, CNN, and Transformer) require signed and non-scaled arithmetic, the testbench is configured to use **bipolar, non-scaled** matrix multiplication across all four designs.

The matrix multiplication evaluated in this work is defined as

$$\mathbf{Y} = \mathbf{X}\mathbf{W}, \quad (1)$$

where $\mathbf{X} \in \mathbb{R}^{B \times F_{in}}$, $\mathbf{W} \in \mathbb{R}^{F_{in} \times F_{out}}$, and $\mathbf{Y} \in \mathbb{R}^{B \times F_{out}}$ stands for the input, weight, and output respectively, with B means the batch size and F_{in} , F_{out} means the input and output feature respectively.

2.1 uGEMM Architecture

uGEMM [1] is a unified unary GEMM architecture that supports both bipolar and unipolar formats, both rate and temporal encoding for multiplication, and both scaled and non-scaled

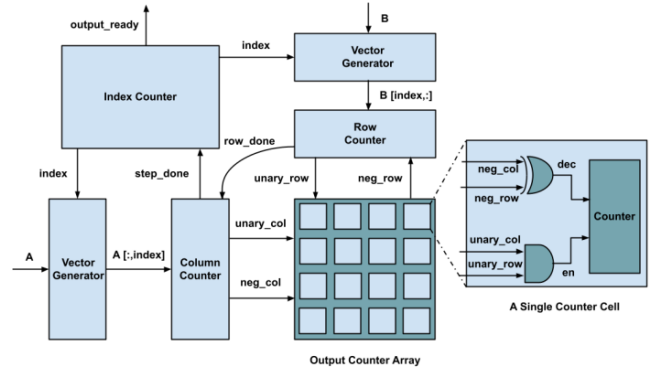


Figure 2: tuGEMM architecture [2]

addition, as shown in Fig. 1. The inputs may be either rate- or temporal-encoded bitstreams, while the output is always a rate-encoded unary bitstream. uGEMM performs approximate computation, meaning its results are not bit-exact with respect to the binary reference. Each operand is encoded over a full bitstream of length 2^w , where w is the binary bit-width.

For uGEMM, computing a single output element of (1) requires F_{in} independent two-operand bipolar multiplications, followed by an F_{in} -input parallel bipolar non-scaled addition.

2.2 tuGEMM Architecture

tuGEMM [2] is a temporal encoded GEMM architecture that supports binary interfaced input and output. tuGEMM supports both signed and unsigned binary inputs, and is bit-exact with respect to the binary reference, as shown in Fig. 2. tuGEMM implements temporal encoded outer-product to perform matrix multiplication.

It takes F_{in} steps to compute the result, according to (1), where each step takes up to $(2^{w-1})^2$ cycles in the worst case, since both column and row elements are temporal encoded. This can be optimized by parallelizing the F_{in} steps into one step, so the max latency becomes $(2^{w-1})^2$ cycles. The testbench utilizes the parallel design.

2.3 tubGEMM Architecture

tubGEMM [3] is a temporal-unary and binary encoded GEMM architecture that supports binary interfaced input and output. tubGEMM supports both signed and unsigned binary inputs, and is bit-exact with respect to the binary reference, similar to tuGEMM. tubGEMM also uses outer-product to perform matrix multiplication. It utilizes two important optimizations

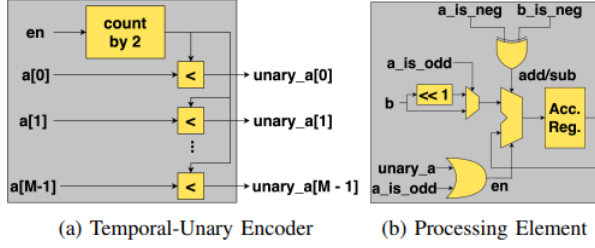


Figure 3: tubGEMM components [3]

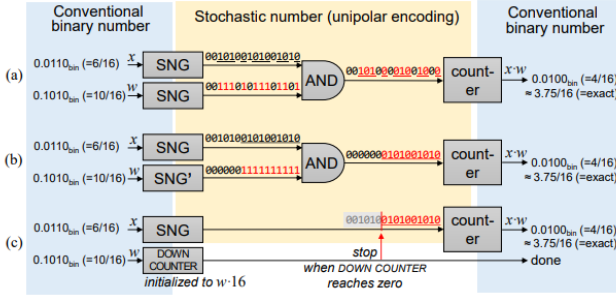


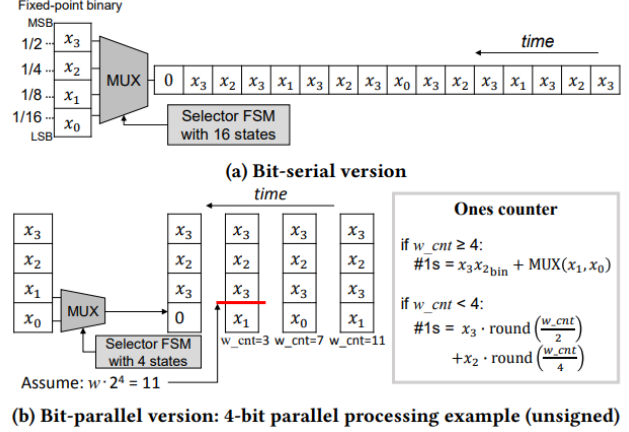
Figure 4: BISC-MVM SC multiplication [4]

for latency: Twos-Unary encoding and binary row addition, as shown in Fig. 3.

With Twos-Unary encoding, each unary cycle represents a magnitude of 2, halving the worst-case multiplication latency to 2^{w-2} cycles for bipolar inputs. With binary row addition, only one column elements are temporal-unary encoded while the other stays in binary, so the latency depends only on the temporal-unary operand. The overall worst-case latency for the matrix multiplication in (1) becomes $F_{in} \cdot 2^{w-2}$ cycles per output element.

2.4 BISC-MVM Architecture

BISC-MVM [4] is a binary-interfaced, unary-encoded GEMM architecture that supports binary interfaced input and output. BISC-MVM supports both signed and unsigned binary inputs, and is an approximate computing design (not bit-exact with respect to the binary reference). Two designs are utilized in BISC-MVM for latency optimization: up-down counter (for signed inputs), as shown in Fig. 4, and bit-parallel SNG, as shown in Fig. 5. The degree of bit-parallelism is 4 in the simulator testbench. For a single scalar-vector multiplication $w\vec{x}$, the latency depends on the magnitude of the shared weight w , taking $\lceil |2^{w-1}w|/b \rceil$ cycles, where b is the degree of bit-parallelism. For a weight matrix, BISC-MVM accumulates F_{in} such scalar-vector multiplications by feeding a sequence of w_i and $\vec{x}_i$, so the total latency per output element of (1) becomes $\sum_{i=1}^{F_{in}} \lceil |2^{w-1}w_i|/b \rceil$ cycles.



(b) Bit-parallel version: 4-bit parallel processing example (unsigned)

Figure 5: Low-discrepancy code generation using FSM [4]

3 Results

The testbench is built on top of UnarySim [6], with my modifications documented in [7]. It consists of two parts: unit tests measuring the latency of a single matrix multiplication for each of the four kernels (uGEMM, tuGEMM, tubGEMM, BISC-MVM), and an integrated test for sparsity sweeps. All experiments use 8-bit precision and 16×16 signed, unscaled input and weight matrices.

3.1 Sparsity Definitions

Three sparsity metrics are defined: matrix-wise, bit-wise, and column-wise. In each sweep, only the target metric is changed while the other two remain constant, so the observed change belongs to that specific type of sparsity.

Matrix-wise sparsity is the fraction of zero elements in the matrix:

$$s_m = \frac{\text{\# of zero elements}}{M \cdot N}. \quad (2)$$

Bit-wise sparsity measures the magnitude of elements instead of how many zeros there are. For w -bit integers, elements are drawn uniformly from $[-cap, cap]$ (bipolar) or $[0, cap]$ (unipolar), where

$$cap = (1 - s_b) \cdot (2^w - 1). \quad (3)$$

A larger s_b shrinks the value range, forcing more leading zero bits per element while leaving most elements nonzero.

Column-wise sparsity is the fraction of all-zero columns:

$$s_c = \frac{\text{\# of all-zero columns}}{N}. \quad (4)$$

It is designed for where the zeros are, not how many or how large.

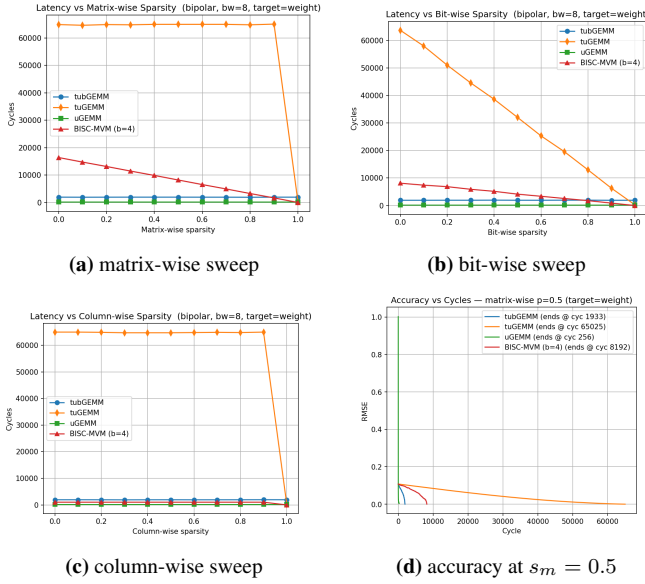


Figure 6: Latency and accuracy results for weight-matrix sparsity sweeps.

3.2 Outcome

In the testbench, when a metric is being swept, other metrics stay unchanged. For example, when sweeping for column-wise sparsity, the matrix-wise and bit-wise will stay unchanged.

For the weight sparsity sweep (Fig. 6), BISC-MVM’s latency decreases linearly with both matrix-wise and bit-wise sparsity, but is insensitive to column-wise sparsity. tuGEMM’s latency decreases linearly with bit-wise sparsity only. tubGEMM and uGEMM are insensitive to all three sparsity types, maintaining a constant latency, with uGEMM having the shortest overall latency.

For the input sparsity sweep (Fig. 7), tubGEMM’s latency decreases linearly with both bit-wise and column-wise sparsity, but is insensitive to matrix-wise sparsity. tuGEMM’s latency again decreases linearly with bit-wise sparsity. uGEMM and BISC-MVM are insensitive to all three sparsity types, with uGEMM again having the shortest overall latency.

Table 1 summarizes above.

Table 1: Sparsity sensitivity of each design. ✓ indicates a linear latency reduction with increasing sparsity; — indicates no sensitivity.

Design	Weight			Input		
	matrix	bit	column	matrix	bit	column
uGEMM	—	—	—	—	—	—
tuGEMM	—	✓	—	—	✓	—
tubGEMM	—	—	—	—	✓	✓
BISC-MVM	✓	✓	—	—	—	—

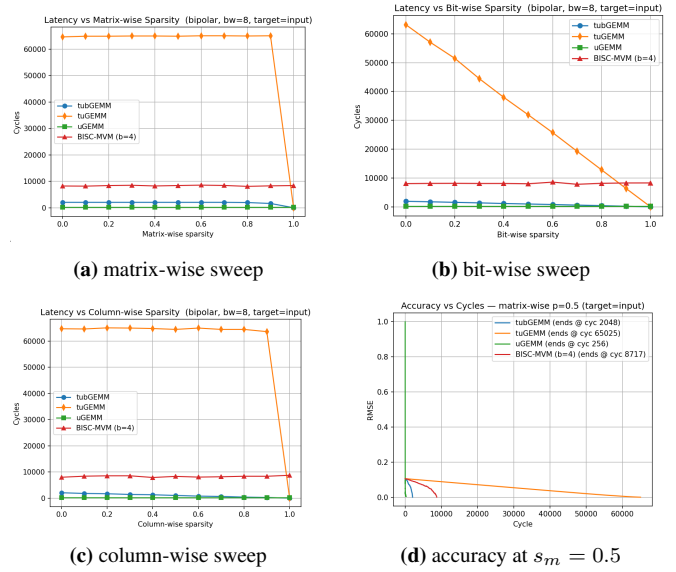


Figure 7: Latency and accuracy results for input-matrix sparsity sweeps.

4 Evaluation and Future Work

The sensitivity pattern in Table 1 can be explained by looking at the latency formula of each design.

uGEMM is a pure unary design with no early termination implemented in the testbench, so every multiplication runs the full bitstream.

$$L_{\text{uGEMM}} = 2^w. \quad (5)$$

uGEMM latency does not change with any sparsity type, for either weight or input.

tuGEMM’s latency is decided by the largest operand product across all the outer-product operations:

$$L_{\text{tuGEMM}}(i, j) = \max_{k \in [1, F_{\text{in}}]} (|X_{ik}| \cdot |W_{kj}|). \quad (6)$$

Bit-wise sparsity makes the max product smaller, so the latency decreases. Matrix-wise and column-wise sparsity don’t change the max, so they don’t help.

tubGEMM’s latency depends on the sum of the column-max across F_{in} columns of the input matrix:

$$L_{\text{tubGEMM}}(i, j) = \sum_{k=1}^{F_{\text{in}}} \max_m |X_{mk}|. \quad (7)$$

column-wise input sparsity helps, because the max value at the column will be 0. Bit-wise input sparsity makes each column-max smaller, which also helps. Matrix-wise sparsity doesn’t change the column-max, so doesn’t help.

BISC-MVM’s latency is the sum of all weight magnitudes in the column, divided by the bit-parallelism b :

$$L_{\text{BISC-MVM}}(i, j) = \sum_{k=1}^{F_{\text{in}}} \left\lceil \frac{|W_{kj}|}{b} \right\rceil. \quad (8)$$

matrix-wise sparsity decreases the sum of weight elements by introducing more zeros, so it helps with latency. Bit-wise

weight sparsity decreases the value of each weight elements directly, so it helps with latency. Column-wise sparsity doesn't help because the total magnitude matters, not where the zeros are.

The current popular sparsity metrics are limited to matrix-wise and bit-wise, which is still a relatively coarse classification. More metrics can be proposed for different use cases. For example, how tubGEMM exploits input matrix column-wise sparsity suggests that other design-specific sparsity metrics should also be considered. In the future, I can explore real datasets and see whether different sparsity metrics can be applied, and update the testbench to run real CNN or MLP models with examining specific sparsity type matrices.

5 Conclusion

This report studied four unary GEMM designs (uGEMM, tuGEMM, tubGEMM, and BISC-MVM) with a focus on how different types of sparsity affect their latency and accuracy under bipolar, non-scaled matrix multiplication. A Python simulator was built based on UnarySim to run sparsity sweeps across all four designs, with three sparsity metrics defined: matrix-wise, bit-wise, and column-wise, for both input and weight matrices.

According to the Results section, each GEMM architecture has its own optimization for one or two of the three sparsity types, which opens up the possibility of pairing a specific design with a specific sparsity pattern. This also bring potential in research for more sparsity metrics, just like column-wise sparsity proposed in the report. With more metrics, real-world workloads can be matched with a design-specific GEMM unit that optimizes for its unique sparsity pattern, just like tubGEMM is optimized for datasets with column-wise sparsity.

6 Tries to Reproduce

Part of the work also focused on recreating the 3-layer MLP for MNIST classification implemented in [1]. The code skeleton was from the original UnarySim repository [6]. The expected result was successfully reproduced for the 14×14 input image size with layer width 64, but not for the 32×32 input size with layer width 512. As shown in Fig. 8, the 14×14 configuration shows a much better early-termination feature and lower RMSE in each fully-connected layer's numerical output.

As an important implementation to test GEMM unit performance, this problem remains unsolved on my side. The following approaches were tried during debugging:

1. Matched binary and unary quantization settings. *Result: error stays the same.*
2. Matched ReLU/Linear module depth against the original implementation. *Result: matched as in the original work.*

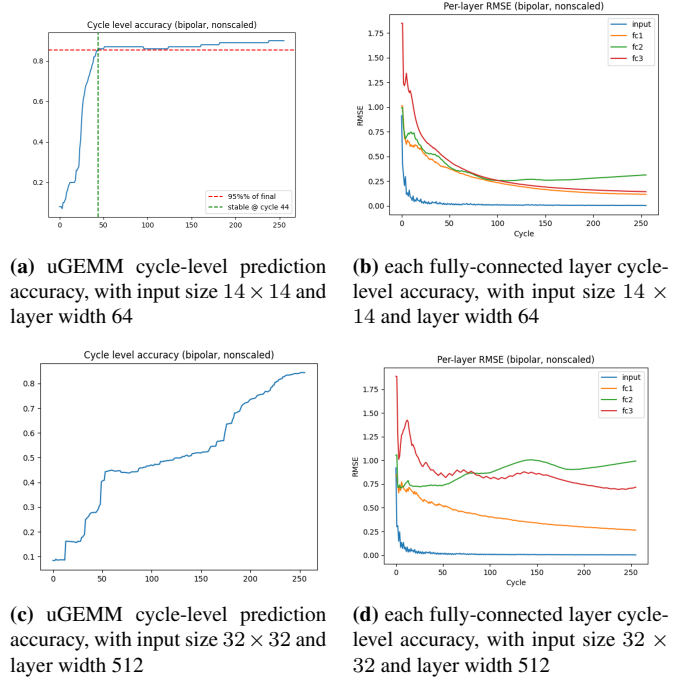


Figure 8: MLP3 Result

3. Cross-checked all properties (bit-width, depth, trained model) against the original implementation. *Result: all matched.*
4. Plotted PP for every layer and compared the pattern with the result. *Result: linear RMSE converges to 0.4, so numerical calculation is also wrong.*
5. Checked PE monitor input, output, and scaling process. *Result: expected output is wrong because MatMul is not clamping.*
6. Tried to use the original kernel and environment. *Result: applied a clamp; the standalone Linear test was solved, but the full MLP was still not.*
7. Decreased layer_width and MNIST_shape to reduce intermediate entry magnitudes. *Result: output is better, but the issue is not fully resolved.*
8. Fixed test_linear quantization. *Result: the standalone Linear test result is better.*
9. Checked the clipper module in the MLP. *Result: the clipper itself has no problem.*
10. Re-quantized the input image to $[-1, 1]$ at 8 bits to test whether image quantization was the cause. *Result: still not working.*
11. Suspected the unary ReLU may be broken at larger value ranges. *Result: binary ReLU does not work in the unary path, so unary ReLU is the only option.*

References

- [1] D. Wu, J. Li, R. Yin, H. Hsiao, Y. Kim, and J. San Miguel, “ugemm: Unary computing architecture for gemm applications,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- [2] H. Nair, P. Vellaisamy, A. Chen, J. Finn, A. Li, M. Trivedi, and J. P. Shen, “tugemm: Area-power-efficient temporal unary gemm architecture for low-precision edge ai,” in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023.
- [3] P. Vellaisamy, H. Nair, J. Finn, M. Trivedi, A. Chen, A. Li, T.-H. Lin, P. H. Wang, R. S. Blanton, and J. P. Shen, “tubgemm: Energy-efficient and sparsity-effective temporal-unary-binary based matrix multiply unit,” in *2023 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2023.
- [4] H. Sim and J. Lee, “A new stochastic computing multiplier with application to deep convolutional neural networks,” in *Proceedings of the 54th Annual Design Automation Conference (DAC)*, 2017.
- [5] P. Vellaisamy, H. Nair, D. Wu, S. Blanton, and J. P. Shen, “Exploration of unary arithmetic-based matrix multiply units for low precision dl accelerators,” in *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2024.
- [6] D. Wu, “UnarySim: A framework for unary computing,” <https://github.com/diwu1990/UnarySim>, 2020.
- [7] Xinyu, “UnarySim,” <https://github.com/xinyuJames/UnarySim>, 2026.